

Object-oriented Programming 2 KU

Spring Boot Introduction

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)

What is Spring Boot?



Spring Boot: Introduction



- Scalable Framework for Microservices
- We will use the Spring Boot & REST system
 - Using **@Annotations** for everything
 - Attach and create typed services easily (vs. Node express.js or Python)

```
@SpringBootApplication
@RestController

public class DemoApplication {
    @GetMapping("/helloworld")
    public String hello() {
        return "Hello World :)";
    }
}
```

Spring Boot: Introduction



- Using @Annotations for **everything**

```

@SpringBootApplication
@RestController

public class DemoApplication {
    @GetMapping("/helloworld")
    public String hello() {
        return "Hello World :)";
    }
}
```



Spring Boot: Introduction



- Spring Boot simplifies the development of stand-alone, production-grade Spring-based applications.
- Key Components
 - **SpringApplication:** Bootstraps and launches the Spring application.
 - **@SpringBootApplication:** Annotation that combines **@Configuration**, **@EnableAutoConfiguration**, and **@ComponentScan**
 - **RESTful APIs:** Use Spring Boot to create and expose REST APIs.

Spring Boot: Key Features



- Auto-Configuration: Automatically configures your application based on dependencies.
- Starter Dependencies: Simplify dependency management.
- Embedded Servers: No need to deploy WAR files; use embedded Tomcat, Jetty, etc.
- Actuator: Provides monitoring and management endpoints.
- Spring Boot CLI: Command-line tool for rapid development.

Spring Boot: Annotations Overview



- `@SpringBootApplication`
 - Entry point for Spring Boot applications.
- `@RestController`
 - `@RestController` is best used when you want to return data (e.g. JSON or XML) rather than a view (HTML). `@RestController` is basically a meta-annotation that combines `@Controller` and `@ResponseBody`
- `@GetMapping` / `@PostMapping`:
 - Maps HTTP GET and POST requests to handler methods.
- `@PathVariable` / `@RequestBody`:
 - Binds method parameters to URL variables and request bodies.



RESTful APIs with Spring Boot



- Principles of REST:
 - Stateless: Each request contains all necessary information.
 - Resource-Based: Manipulate resources via HTTP methods.
 - Representation: Use JSON/XML for data exchange.
- Spring Boot Support:
 - Simplifies creation of RESTful endpoints.
 - Handles JSON serialization/deserialization automatically.

Example: EventLoggerApplication

Project Structure Overview



- This project is a simple Spring Boot application that simulates an event logging system. It provides an API to create and retrieve events using HTTP methods (GET and POST). The events are stored in memory (a list) and assigned unique IDs automatically.
- Main Components:
 - Models (Event, EventRequest)
 - Repository (EventRepository)
 - Controller (EventController)
 - Application (EventLoggerApplication)
- Open the project in IntelliJ and follow along 😊
 - Download the project [here](#)



The Event Model



```
package at.tugraz;

import lombok.Data;
import java.util.Date;

@Data
public class Event {
    private String name;
    private long id;
    private Date time;
    private static long eventCounter = 0L;

    public Event(String name) {
        this.name = name;
        this.time = new Date(System.currentTimeMillis());
        this.id = eventCounter++;
    }
}
```

The Event Model

- Purpose:
 - Defines the Event model with attributes name, id, and time. Each event gets a unique id and a timestamp when created.
- Key Point:
 - Uses @Data from Lombok to auto-generate getters/setters.
 - A static counter eventCounter ensures each event has a unique ID.

```
package at.tugraz;

import lombok.Data;
import java.util.Date;

@Data
public class Event {
    private String name;
    private long id;
    private Date time;
    private static long eventCounter = 0L;

    public Event(String name) {
        this.name = name;
        this.time = new Date(System.currentTimeMillis());
        this.id = eventCounter++;
    }
}
```

The Event Repository



```
package at.tugraz;

import lombok.Data;
import java.util.ArrayList;
import java.util.List;

@Data
public class EventRepository {
    List<Event> events = new ArrayList<>();
    private static EventRepository instance = null;

    public static EventRepository getInstance() {
        if (instance == null) {
            instance = new EventRepository();
        }
        return instance;
    }
}
```

The Event Repository

- Provides an in-memory store for all events using a singleton pattern to ensure only one instance of the repository exists.
- Main Components:
 - Singleton Pattern: Ensures a single instance of EventRepository.
 - events List: Stores Event objects.
 - Lombok @Data: Generates necessary methods.

```
package at.tugraz;

import lombok.Data;
import java.util.ArrayList;
import java.util.List;

@Data
public class EventRepository {
    List<Event> events = new ArrayList<>();
    private static EventRepository instance = null;

    public static EventRepository getInstance() {
        if (instance == null) {
            instance = new EventRepository();
        }
        return instance;
    }
}
```

The Event Controller



```
package at.tugraz;

import org.springframework.web.bind.annotation.*;

@RestController
public class EventController {
    @GetMapping("/events")
    EventRepository events() {
        return EventRepository.getInstance();
    }

    @GetMapping("/events/{id}")
    Event event(@PathVariable long id) {
        return EventRepository.getInstance().getEvents().stream()
            .filter(evt -> evt.getId() == id)
            .findFirst()
            .orElseThrow(RuntimeException::new);
    }

    @PostMapping("/events")
    Event event(@RequestBody(required = true) EventRequest evt) {
        var event_created = new Event(evt.getName());
        EventRepository.getInstance().getEvents().add(event_created);
        return event_created;
    }
}
```

The Event Controller

- Acts as the REST API controller for handling HTTP requests.
- It provides endpoints for creating and retrieving events.
- Annotations:
 - @RestController: Combines @Controller and @ResponseBody.
 - @GetMapping, @PostMapping: Map HTTP methods to handler methods.
 - @PathVariable, @RequestBody: Bind URL parameters and request bodies.

```
package at.tugraz;

import org.springframework.web.bind.annotation.*;

@RestController
public class EventController {
    @GetMapping("/events")
    EventRepository events() {
        return EventRepository.getInstance();
    }

    @GetMapping("/events/{id}")
    Event event(@PathVariable long id) {
        return EventRepository.getInstance().getEvents().stream()
            .filter(evt -> evt.getId() == id)
            .findFirst()
            .orElseThrow(RuntimeException::new);
    }

    @PostMapping("/events")
    Event event(@RequestBody(required = true) EventRequest evt) {
        var event_created = new Event(evt.getName());
        EventRepository.getInstance().getEvents().add(event_created);
        return event_created;
    }
}
```

The Event Controller

- Endpoints:
 - GET /events: Retrieves all events.
 - GET /events/{id}:
Retrieves a specific event by ID.
 - POST /events: Creates a new event.

```
package at.tugraz;

import org.springframework.web.bind.annotation.*;

@RestController
public class EventController {
    @GetMapping("/events")
    EventRepository events() {
        return EventRepository.getInstance();
    }

    @GetMapping("/events/{id}")
    Event event(@PathVariable long id) {
        return EventRepository.getInstance().getEvents().stream()
            .filter(evt -> evt.getId() == id)
            .findFirst()
            .orElseThrow(RuntimeException::new);
    }

    @PostMapping("/events")
    Event event(@RequestBody(required = true) EventRequest evt) {
        var event_created = new Event(evt.getName());
        EventRepository.getInstance().getEvents().add(event_created);
        return event_created;
    }
}
```

The Event Request Model



- Represents the payload for creating a new event.
- Again – Lombok @Data annotation.



```
package at.tugraz;  
  
import lombok.Data;  
  
@Data  
public class EventRequest {  
    private String name;  
}
```

Application Entry Point



```
package at.tugraz;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import java.util.Collections;

@SpringBootApplication
public class EventLoggerApplication {

    public static void main(String... args) {
        var app = new SpringApplication(EventLoggerApplication.class);
        app.setDefaultProperties(Collections.singletonMap("server.port", 8020));
        app.run();
    }
}
```

Application Entry Point



```
package at.tugraz;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import java.util.Collections;

@SpringBootApplication
public class EventLoggerApplication {

    public static void main(String... args) {
        var app = new SpringApplication(EventLoggerApplication.class);
        app.setDefaultProperties(Collections.singletonMap("server.port", 8020));
        app.run();
    }
}
```

- @SpringBootApplication: Combines @Configuration, @EnableAutoConfiguration, and @ComponentScan.
- Main Method: Bootstraps the application.
- Custom Server Port: Sets server port to 8020 using setDefaultProperties.

Testing Requests



- Insomnia:
 - Download Insomnia [here](#)
 - Watch the live demonstration for this sample code from last year [here](#)
- Postman
 - Download Postman [here](#)



POSTMAN



Insomnia

Next week:
Environmental Variables
Error Handling in Spring Boot
(Maybe) More hints for A0



Object-oriented Programming 2 KU

Spring Boot Introduction

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)